1️⃣ What is SharedPreferences?

SharedPreferences is used to store **small data locally** in the device as **key–value pairs**.

📌 Examples of data:

- Login status (`true / false`)
- Username
- Theme mode (dark/light)
- App settings

✅ Data stays **even after app restart**

❌ Not for large or sensitive data

2️⃣ What is `prefs`?

```
final prefs = await SharedPreferences.getInstance();
```

🔍 Meaning:

- `prefs` is just a **variable name**
- It holds the **SharedPreferences object**
- Through `prefs`, we can:
  - Save data
  - Read data
  - Delete data

📦 Think of `prefs` as a **local storage box**

3️⃣ What is `async`?

```
Future<void> saveData() async {
  ...
}
```

🔍 Meaning:

- `async` means:

  "This function takes time to complete"

Why needed?

- Getting SharedPreferences is **not instant**
- Flutter must wait until data is loaded from storage

Without `async` ❌
With `async` ✅ Flutter waits properly

4️⃣ What is `await`?

```
final prefs = await SharedPreferences.getInstance();
```

🔍 Meaning:

- `await` means:

  "Wait here until this work finishes"

Example:

- Flutter waits until storage is ready

- Then continues execution

🚦 Without `await`, app may crash or return null data

5️⃣ What is `Future`?

```
Future<void> saveData()
```

🔍 Meaning:

- `Future` = value that comes **later**
- `void` = returns nothing

Other examples:

```
Future<String> getName()
Future<bool> isLoggedIn()
```

6️⃣ Saving data – function explanation

```
Future<void> saveName(String name) async {
  final prefs = await SharedPreferences.getInstance();
  await prefs.setString('username', name);
}
```

🔎 Line-by-line:

Line	Meaning
`Future<void>`	Function runs later, returns nothing
`async`	Allows using `await`
`getInstance()`	Opens local storage

`setString()`	Saves string data
`'username'`	Key (like label)
`name`	Value to save

📦 Saved as:
`username = "Rahul"`

7️⃣ Reading data – function explanation

```
Future<String> loadName() async {
  final prefs = await SharedPreferences.getInstance();
  return prefs.getString('username') ?? "No name saved";
}
```

🔎 Explanation:

Part	Meaning
`getString()`	Reads saved value
`'username'`	Same key used to save
`??`	Default value if null
`"No name saved"`	Shown if nothing saved

8️⃣ ?? (Null-coalescing operator)

```
prefs.getString('username') ?? "No name saved";
```

🔍 Meaning:

If value is `null`, use the right side

Example:

- If name exists → show name
- If not → show default text

9️⃣ Why call in `initState()`?

```
@override
void initState() {
  super.initState();
  loadName();
}
```

🔍 Meaning:

- `initState()` runs when screen loads
- Perfect place to:
  - Load saved data
  - Restore state after restart

🔄 Complete mental flow (VERY IMPORTANT)

```
App starts
   ↓
initState() runs
   ↓
loadName() called
   ↓
SharedPreferences opened
   ↓
Saved value read
   ↓
Value shown on UI
```

🔑 Common SharedPreferences functions

Function	Use
`setString()`	Save string
`setBool()`	Save true/false
`setInt()`	Save number
`getString()`	Read string
`remove()`	Delete one value
`clear()`	Delete all

```
import 'package:flutter/material.dart';

import 'package:shared_preferences/shared_preferences.dart';


void main() {

  runApp(const MyApp());

}


class MyApp extends StatelessWidget {

  const MyApp({super.key});


  @override

  Widget build(BuildContext context) {

    return MaterialApp(

      debugShowCheckedModeBanner: false,

      home: const HomeScreen(),

    );
```

```
  }
}

class HomeScreen extends StatefulWidget {
  const HomeScreen({super.key});

  @override
  State<HomeScreen> createState() => _HomeScreenState();
}

class _HomeScreenState extends State<HomeScreen> {
  final TextEditingController _controller = TextEditingController();
  String savedName = "";

  @override
  void initState() {
    super.initState();
    loadName(); // 👈 Load saved value on app start
  }

  /// Load value from SharedPreferences
  Future<void> loadName() async {
    final prefs = await SharedPreferences.getInstance();
    setState(() {
      savedName = prefs.getString('username') ?? "No name saved";
    });
```

```
}

/// Save value to SharedPreferences
Future<void> saveName() async {
  final prefs = await SharedPreferences.getInstance();
  await prefs.setString('username', _controller.text);

  setState(() {
    savedName = _controller.text;
  });
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(title: const Text("SharedPreferences Demo")),
    body: Padding(
      padding: const EdgeInsets.all(16),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          TextField(
            controller: _controller,
            decoration: const InputDecoration(
              labelText: "Enter your name",
              border: OutlineInputBorder(),
```

```
            ),
          ),
          const SizedBox(height: 12),
          ElevatedButton(
            onPressed: saveName,
            child: const Text("Save"),
          ),
          const SizedBox(height: 20),
          const Text(
            "Saved value:",
            style: TextStyle(fontSize: 16, fontWeight: FontWeight.bold),
          ),
          const SizedBox(height: 8),
          Text(
            savedName,
            style: const TextStyle(fontSize: 18, color: Colors.blue),
          ),
        ],
      ),
    ),
  );
}
}
```

CRUD OPERATIONS

```
import 'package:flutter/material.dart';
import 'package:shared_preferences/shared_preferences.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
      debugShowCheckedModeBanner: false,
      home: TodoPage(),
    );
  }
}

class TodoPage extends StatefulWidget {
  const TodoPage({super.key});

  @override
  State<TodoPage> createState() => _TodoPageState();
}

class _TodoPageState extends State<TodoPage> {
  // ================= VARIABLES =================
  final TextEditingController _controller = TextEditingController();
  List<String> todos = [];

  // ================= INIT =================
  @override
  void initState() {
    super.initState();
    loadTodos(); // READ
```

```
}

// ================= CRUD FUNCTIONS =================

/// READ – load todos from storage
Future<void> loadTodos() async {
  final prefs = await SharedPreferences.getInstance();
  todos = prefs.getStringList('todos') ?? [];
  setState(() {});
}

/// CREATE – add new todo
Future<void> addTodo() async {
  if (_controller.text.isEmpty) return;

  final prefs = await SharedPreferences.getInstance();
  todos.add(_controller.text);
  await prefs.setStringList('todos', todos);

  _controller.clear();
  setState(() {});
}

/// UPDATE – edit existing todo
Future<void> updateTodo(int index) async {
  final prefs = await SharedPreferences.getInstance();
  todos[index] = _controller.text;
  await prefs.setStringList('todos', todos);

  _controller.clear();
  setState(() {});
}

/// DELETE – remove todo
Future<void> deleteTodo(int index) async {
  final prefs = await SharedPreferences.getInstance();
  todos.removeAt(index);
  await prefs.setStringList('todos', todos);

  setState(() {});
}

// ================= UI HELPERS =================
```

```
void showEditDialog(int index) {
  _controller.text = todos[index];

  showDialog(
    context: context,
    builder: (_) => AlertDialog(
      title: const Text("Edit Todo"),
      content: TextField(controller: _controller),
      actions: [
        TextButton(
          onPressed: () {
            updateTodo(index);
            Navigator.pop(context);
          },
          child: const Text("Update"),
        ),
      ],
    ),
  );
}

// ================= UI =================
@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(title: const Text("Simple Todo CRUD")),
    body: Column(
      children: [
        Padding(
          padding: const EdgeInsets.all(8),
          child: TextField(
            controller: _controller,
            decoration: const InputDecoration(
              hintText: "Enter todo",
              border: OutlineInputBorder(),
            ),
          ),
        ),
        ElevatedButton(
          onPressed: addTodo,
          child: const Text("Add Todo"),
        ),
        Expanded(
          child: ListView.builder(
```

```
            itemCount: todos.length,
            itemBuilder: (_, index) => ListTile(
              title: Text(todos[index]),
              trailing: Row(
                mainAxisSize: MainAxisSize.min,
                children: [
                  IconButton(
                    icon: const Icon(Icons.edit),
                    onPressed: () => showEditDialog(index),
                  ),
                  IconButton(
                    icon: const Icon(Icons.delete),
                    onPressed: () => deleteTodo(index),
                  ),
                ],
              ),
            ),
          ),
        ),
      ],
    ),
  );
}
}
```

6️⃣ Variables Section

```
final TextEditingController _controller =
TextEditingController();
List<String> todos = [];
```

🔹 **TextEditingController**

- Controls the `TextField`
- Used to:
  - Read input

    - Clear input
    - Set text (for edit)

🔹 **`List<String> todos`**

- Stores all todo items

Example:

```
["Buy milk", "Learn Flutter"]
```

-

7️⃣ initState() – Runs once when screen opens

```
@override
void initState() {
  super.initState();
  loadTodos(); // READ
}
```

🔹 **What happens?**

- Called **only once**
- Loads saved todos from phone storage
- This is why todos appear **after app restart**

8️⃣ CRUD FUNCTIONS (CORE LOGIC)

🟢 READ – Load Todos

```
Future<void> loadTodos() async {
```

🔹 **Why `Future` + `async`?**

- SharedPreferences works asynchronously
- Takes time to read data

```
final prefs = await SharedPreferences.getInstance();
```

- Gets access to local storage
- `await` → wait until storage is ready

```
todos = prefs.getStringList('todos') ?? [];
```

- Reads list stored with key `"todos"`
- If nothing exists → empty list `[]`

```
setState(() {});
```

- Tells Flutter:

  "Data changed, rebuild UI"

🟢 CREATE – Add Todo

```
Future<void> addTodo() async {
```

```
if (_controller.text.isEmpty) return;
```

- Prevents adding empty todo

```
final prefs = await SharedPreferences.getInstance();
todos.add(_controller.text);
```

- Adds text to todo list

```
await prefs.setStringList('todos', todos);
```

- Saves updated list to storage

```
_controller.clear();
setState(() {});
```

- Clears input field
- Refreshes UI

🟡 UPDATE – Edit Todo

```
Future<void> updateTodo(int index) async {
```

- `index` → position of todo in list

```
todos[index] = _controller.text;
```

- Replaces old todo with new text

```
await prefs.setStringList('todos', todos);
```

- Saves updated list

```
_controller.clear();
setState(() {});
```

- Clears field
- Updates UI

🔴 DELETE – Remove Todo

```
Future<void> deleteTodo(int index) async {
```

```
todos.removeAt(index);
```

- Removes selected todo

```
await prefs.setStringList('todos', todos);
```

- Updates storage

```
setState(() {});
```

- Rebuilds UI

9️⃣ Edit Dialog UI

```
void showEditDialog(int index) {
```

```
_controller.text = todos[index];
```

- Loads selected todo into text field

```
showDialog(
  context: context,
```

- Opens popup dialog

```
AlertDialog(
  title: const Text("Edit Todo"),
  content: TextField(controller: _controller),
```

- Dialog with input field

```
onPressed: () {
  updateTodo(index);
  Navigator.pop(context);
},
```

- Updates todo
- Closes dialog

🔟 UI (build method)

```
@override
Widget build(BuildContext context) {
```

🔹 Scaffold

- Basic screen structure

```
Scaffold(
  appBar: AppBar(title: const Text("Simple Todo CRUD")),
```

🔹 Input Field

```
TextField(
  controller: _controller,
  decoration: InputDecoration(
    hintText: "Enter todo",
```

- User enters todo here

🔹 Add Button

```
ElevatedButton(
  onPressed: addTodo,
```

- Calls **CREATE** function

🔹 Todo List

```
ListView.builder(
```

```
  itemCount: todos.length,
```

- Builds list dynamically

```
title: Text(todos[index]),
```

- Displays todo text

🔹 **Edit & Delete Buttons**

```
IconButton(
  icon: Icon(Icons.edit),
```

- Opens edit dialog

```
IconButton(
  icon: Icon(Icons.delete),
```

- Deletes todo

🔁 App Flow (VERY IMPORTANT)

1. App starts
2. `initState()` runs
3. Todos loaded from storage
4. UI updates
5. User adds/edits/deletes

6. Data saved in SharedPreferences
7. App restarts → data still exists ✅